

1. INTRODUCTION

1.1 Brief Information about the Project

A Hotel Reservation Management System is a database application that manages hotel information and administrative processes. The project provides a centralized relational database for guests, room types, rooms, reservations, payments, and hotel services. It is intended to replace fragmented manual records and multiple spreadsheet files with a structured database that supports consistent storage and efficient retrieval.

1.2 Motivation and Contribution of Project

The project is motivated by common problems in hotel record management such as duplicate records, time-consuming searches, inconsistent information, and difficulty in generating reports. A centralized database provides a systematic approach to storing related hotel data and enforcing relationships through keys and constraints.

- Centralizing hotel records in one relational database.
- Reducing duplicate and inconsistent information.
- Maintaining referential integrity using primary and foreign keys.
- Supporting faster hotel queries and report generation.
- Providing a foundation for guest, staff, room, reservation, and payment management.

1.3 Objectives of the Project

- Develop a centralized hotel database.
- Manage guest records.
- Maintain staff information.
- Store room type details.
- Handle room registration and guest reservations.
- Record examination payments.
- Reduce duplicate data and update anomalies.
- Maintain referential integrity.
- Support efficient SQL queries and hotel reports.

1.4 Scope of the Project

The scope of the project covers the database-oriented management of hotel reservation information. The core scope includes managing hotel, room, guest, staff, reservation, and payment data. The system supports storing, updating, retrieving, and maintaining these records in an organized database. It also includes room availability, reservation, check-in and check-out, and payment-related information. The database can serve as the backend for a hotel management application and can be extended in the future with additional features such as online booking, automated notifications, customer feedback, billing, reporting, and web/mobile interfaces.

2. SYSTEM ANALYSIS

2.1 Existing System

- Manual record maintenance.
- Multiple Excel sheets or disconnected files.
- Duplicate records and inconsistent information.
- Time-consuming searches.
- Difficulty in generating consolidated reports.
- Higher possibility of data-entry errors.

2.2 Proposed System

The proposed system uses a centralized relational database. It organizes hotel information into related tables and uses primary keys, foreign keys, uniqueness rules, NOT NULL conditions, and CHECK constraints where appropriate. The system supports guest, staff, room type, room, reservation, and payment management.

- Centralized database
- Guest management
- Staff management
- Room management
- Reservation management
- Payment processing
- SQL-based reporting
- Referential integrity

2.3 Functional Requirements

Module	Functional Requirement
Hotel Management	Add, update, search, and display hotel details.
Room Management	Store and manage room details, room types, prices, and hotel associations..
Guest Management	Store and maintain guest details and contact information.
Staff Management	Store staff details and manage their roles and hotel associations.
Reservation Management	Create, update, search, and manage guest room reservations, including check-in, check-out, and reservation status.
Payment Management	Record and manage payment amount, payment date, payment status, and reservation details.
Search and Reporting	Retrieve hotel, room, guest, reservation, staff, and payment information using SQL queries.

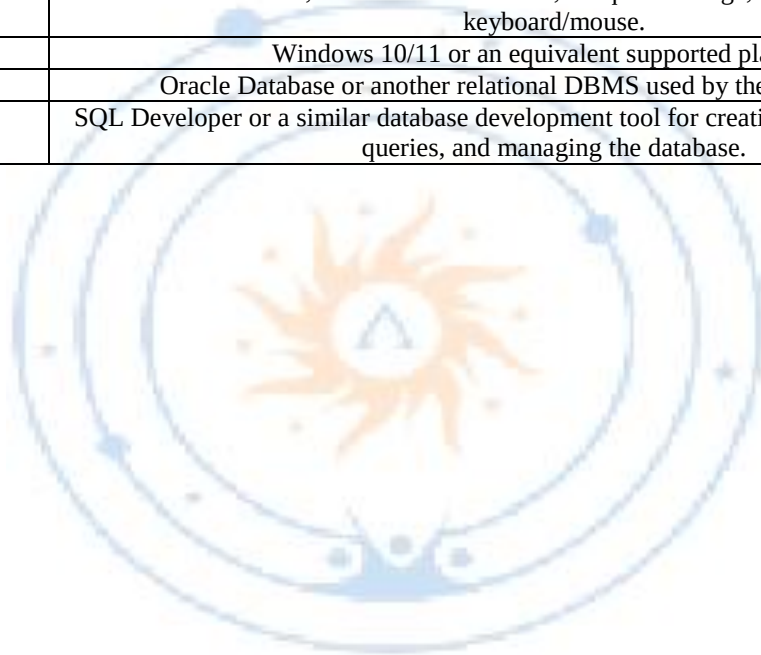
2.4 Non-Functional Requirements

- **Performance** – Queries should retrieve hotel, room, guest, reservation, and payment information efficiently for normal hotel workloads.

- **Integrity** – Invalid foreign-key references and duplicate primary-key values should be prevented to maintain accurate and consistent data.
- **Security** – Customer and payment information should be protected, and database privileges should be controlled for authorized administrative users.
- **Usability** – The system should provide simple and understandable data entry, search, and reporting functions for hotel users.
- **Reliability** – Database transactions should maintain a consistent database state and prevent incorrect or incomplete records.
- **Scalability** – The database design should support growth in hotels, rooms, guests, reservations, staff, and payment records.

2.5 Requirements Specification

Category	Suggested Requirement
Hardware	Intel i3 or above; 4 GB RAM minimum; adequate storage; standard display and keyboard/mouse.
Operating System	Windows 10/11 or an equivalent supported platform.
Database	Oracle Database or another relational DBMS used by the implementation.
Software / Tools	SQL Developer or a similar database development tool for creating tables, executing SQL queries, and managing the database.



3. DATABASE DESIGN

3.1 Major Entities

Entity	Description
Hotel	Stores hotel information such as hotel name, location, and contact details.
Room	Stores room details such as room type, price, and hotel association.
Guest	Stores guest details such as name, phone, and email.
Staff	Stores information about hotel staff members and their roles.
Reservation	Stores reservation details such as check-in, check-out, status, guest, and room information.
Payment	Stores payment details such as amount, payment date, payment status, and reservation information..

3.2 ER Model

The ER model represents the major entities and their relationships in the Hotel Reservation Management System. The main entities identified in the project are Hotel, Room, Guest, Staff, Reservation, and Payment. The relationships include Hotel–Room, Hotel–Staff, Guest–Reservation, Room–Reservation, and Reservation–Payment. The ER model also includes attributes such as Hotel_ID, Hotel_Name, Location, Room_ID, Room_Type, Price, Guest_ID, Guest_Name, Staff_ID, Reservation_ID, Check_In, Check_Out, Payment_ID, Amount, and Payment_Status. The relationships and cardinalities help represent how hotels, rooms, guests, staff, reservations, and payments are connected in the database.

3.3 Relationships

Relationship	Description
Hotel → Room	One hotel can contain many rooms.
Hotel → Staff	One hotel can employ many staff members.
Guest → Reservation	One guest can make multiple reservations.
Room → Reservation	One room can have multiple reservations over different periods.
Reservation → Payment	One reservation has one payment record.

4. ER DIAGRAM

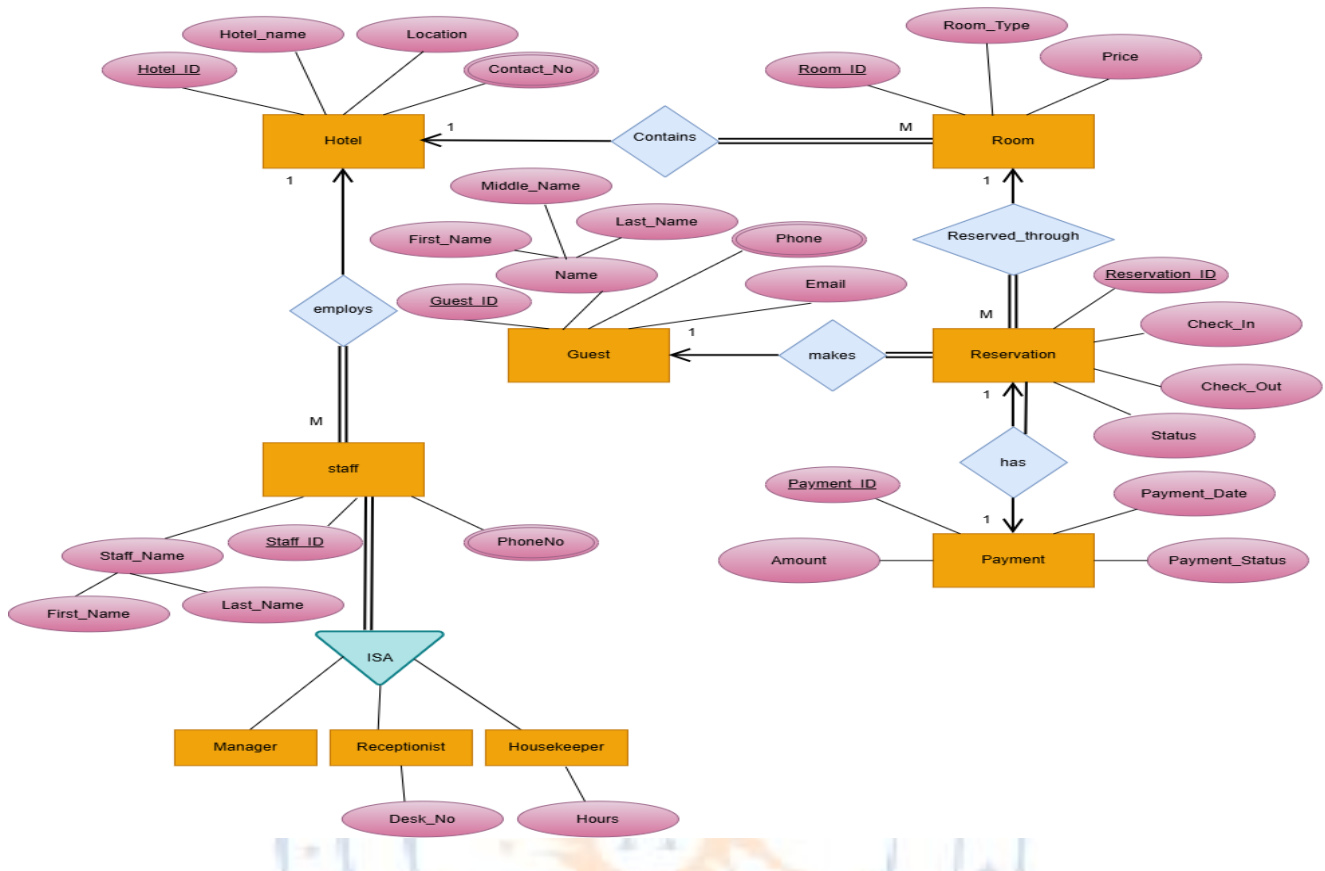


Figure 1: ER Diagram of Hotel Reservation Management System

The ER diagram summarizes the main entities, relationships, and cardinalities of the Hotel Reservation Management System. It represents the connections between Hotel, Room, Guest, Staff, Reservation, and Payment entities and provides a clear documentation-level view of the database structure. The diagram is based on the database design described in the project and reflects the relationships defined for the implemented system.

5. RELATIONAL SCHEMA

Relation	Relational Schema
Hotel	Hotel_ID (PK), Hotel_Name, Location, Contact_No
Room	Room_ID (PK), Room_Type, Price, Hotel_ID (FK)
Guest	Guest_ID (PK), Guest_Name, Phone, Email
Staff	Staff_ID (PK), Staff_Name, Role, PhoneNo, Hotel_ID (FK)
Reservation	Reservation_ID (PK), Check_In, Check_Out, Status, Guest_ID (FK), Room_ID (FK)
Payment	Payment_ID (PK), Amount, Payment_Date, Payment_Status, Reservation_ID (FK)

The relational schema follows the structure presented in the project material. The Room relation references Hotel, the Staff relation references Hotel, the Reservation relation references Guest and Room, and the Payment relation references Reservation. These primary-key and foreign-key relationships maintain connections between the different entities and support data integrity in the database.



6. TABLE DESIGN

6.1 Hotel Table

Attribute	Data Type	Constraint	Description
Hotel_ID	INT / NUMBER	PK, NOT NULL	Unique hotel identifier
Hotel_Name	VARCHAR	NOT NULL	Name of the hotel
Location	VARCHAR		Hotel location
Ontact_No	VARCHAR	—	Hotel contact number

6.2 Room Table

Attribute	Data Type	Constraint	Description
Room_ID	INT	PK, NOT NULL	Unique student identifier
Room_Type	VARCHAR	NOT NULL	Student name
Price	REAL / NUMBER		Institution roll number
Hotel_ID	INT / NUMBER	FK	Gender value as required by implementation

6.3 Guest Table

Attribute	Data Type	Constraint	Description
Faculty_ID	INT	PK	Unique faculty identifier
Faculty_Name	VARCHAR	NOT NULL	Faculty name
Qualification	VARCHAR	—	Academic qualification
Designation	VARCHAR	—	Faculty designation
Email	VARCHAR	UNIQUE	Faculty email
Phone_No	VARCHAR	—	Contact number
Join_Date	DATE	—	Joining date
Experience	INT	—	Experience in years, if maintained
Department_ID	INT	FK	References Department

6.4 Staff Table

Attribute	Data Type	Constraint	Description
Staff_ID	INT/ NUMBER	PK	Unique course identifier
Staff_Name	VARCHAR	UNIQUE	Course code
Role	VARCHAR	NOT NULL	Course name
PhoneNo	VARCHAR	CHECK	Credit value
Hotel_ID	INT/ NUMBER	—	Semester offered

6.5 Reservation Table

Attribute	Data Type	Constraint	Description
Reservation_ID	INT/ NUMBER	PK, NOT NULL	Unique RESERVATION identifier
Check_In	DATE	---	Guest check-in date
Check_Out	DATE	---	Guest check-out date
Status	VARCHAR	—	Current reservation status
Guest_ID	INT/ NUMBER	FK	References Guest
Room_ID	INT/ NUMBER	FK	References Room

6.6 payment Table

Attribute	Data Type	Constraint	Description
Payment_ID	INT/ NUMBER	PK, NOT NULL	Unique payment identifier
Amount	REAL/ NUMBER		Payment amount
Payment_Date	DATE	—	Date of payment
Payment_Status	VARCHAR	—	Payment status
Reservation_ID	INT/ NUMBER	FK	References Reservation

6.7 Keys and Constraints

Constraint Type	Applied To / Purpose
Primary Key	Hotel_ID, Room_ID, Guest_ID, Staff_ID, Reservation_ID, Payment_ID
Foreign Key	Room.Hotel_ID; Staff.Hotel_ID; Reservation.Guest_ID; Reservation.Room_ID; Payment.Reservation_ID
NOT NULL	Required attributes such as IDs, names, and other mandatory fields
UNIQUE	Can be applied to identifiers or other attributes that must contain unique values
CHECK	Can be used to restrict values such as Price, Amount, and valid reservation/payment status
DEFAULT	Can be used for fields such as reservation status or payment status when appropriate

7. NORMALIZATION

Normalization organizes data into related relations to reduce data redundancy and avoid insertion, update, and deletion anomalies. In this project, the database is normalized from UNF to 1NF, 2NF, and 3NF. The normalization process separates hotel, room, guest, reservation, and payment information into appropriate relations while maintaining relationships through primary keys and foreign keys.

7.1 Unnormalized Form (UNF)

In UNF, hotel reservation information may be stored in a single relation containing Hotel_ID, Hotel_Name, Location, Room_ID, Room_Type, Price, Guest_ID, Guest_Name, Phone, Reservation_ID, Check_In, Check_Out, and Payment details. Repeated room, guest, reservation, and payment information can create data redundancy and make updates difficult.

7.2 First Normal Form (1NF)

In 1NF, repeating groups are removed and all attributes contain atomic values. Each room and guest combination is represented as a separate reservation record. The 1NF relation used in the project is:

RESERVATION_1NF(Reservation_ID,Room_ID,Guest_ID,Hotel_ID,Payment_ID,
Payment_Amount,Payment_Status)

PRIMARY KEY (Room_ID, Guest_ID)

The functional dependencies are:

RESERVATION_ID → GUEST_ID, ROOM_ID, HOTEL_ID, PAYMENT_ID

PAYMENT_ID → PAYMENT_AMOUNT, PAYMENT_STATUS

ROOM_ID → HOTEL_ID

RESERVATION_ID	GUEST_ID	ROOM_ID	HOTEL_ID	PAYMENT_ID	PAYMENT_AMOUNT	PAYMENT_STATUS
1016	1	101	1	5016	4500	Paid
1017	1	102	1	5017	5000	Paid
1018	2	101	1	5018	4500	Paid
1019	2	103	2	5019	10500	Paid
1020	3	104	2	5020	15000	Pending

7.3 Second Normal Form (2NF)

In 2NF, partial dependencies on the composite primary key are removed. Room-specific details depend on Room_ID, while guest-specific details depend on Guest_ID. Therefore, the relation is decomposed into separate room and reservation relations.

PAYMENT_2NF(PAYMENT_ID,PAYMENT_AMOUNT,PAYMENT_STATUS)

PRIMARY KEY(PAYMENT_ID)

RESERVATION_2NF(RESERVATION_ID,GUEST_ID,ROOM_ID,HOTEL_ID,PAYMENT_ID)

PRIMARY KEY(RESERVATION_ID,GUEST_ID,ROOM_ID,HOTEL_ID,PAYMENT_ID)

Functional Dependencies

PAYMENT_ID → PAYMENT_AMOUNT, PAYMENT_STATUS

RESERVATION_ID → GUEST_ID, ROOM_ID, HOTEL_ID, PAYMENT_ID

ROOM_ID → HOTEL_ID

This separation reduces partial dependency and avoids unnecessary repetition of room information.

PAYMENT:

PAYMENT_ID	PAYMENT_AMOUNT	PAYMENT_STATUS
5016	4500	Paid
5017	5000	Paid
5018	4500	Paid
5019	10500	Paid
5020	15000	Pending

RESERVATION:

RESERVATION_ID	GUEST_ID	ROOM_ID	HOTEL_ID	PAYMENT_ID
1016	1	101	1	5016
1017	1	102	1	5017
1018	2	101	1	5018
1019	2	103	2	5019
1020	3	104	2	5020

7.4 Third Normal Form (3NF)

In 3NF, transitive dependencies are removed by separating hotel-specific information from room-specific information. Hotel details are stored in the HOTEL_3NF relation, while room details are stored in the ROOM_3NF relation. This separation reduces redundancy and improves data consistency.

HOTEL_3NF(Hotel_ID, Room_ID)

PRIMARY KEY(Hotel_ID,Room_ID)

RESERVATION_3NF(Room_ID, Reservation_ID,Payment_ID,Guest_ID)

PRIMARY KEY(Room_ID,Reservation_ID,Payment_ID,Guest_ID)

Relation	Reason for Separation
Hotel_3NF	Stores hotel-specific attributes such as Hotel_ID and Location once.
Room_3NF	stores room-specific attributes and references the corresponding hotel.

RESERVATION:

RESERVATION_ID	GUEST_ID	ROOM_ID	PAYMENT_ID
1016	1	101	5016
1017	1	102	5017
1018	2	101	5018
1019	2	103	5019
1020	3	104	5020

ROOM

ROOM_ID	HOTEL_ID
101	1
102	1
103	2
104	2

8. SQL IMPLEMENTATION

8.1 DDL – Data Definition Language

The following DDL statements provide a documentation-ready implementation of the Hotel Reservation Management System based on the relational schema. The tables, primary keys, foreign keys, and other constraints are defined according to the database design. Data types can be adjusted according to the specific DBMS used for implementation.

```
create table Hotel

( Hotel_ID integer,

Hotel_name varchar(100),

Location varchar(100),

Contact_no varchar(100),

constraint pk_Hotel primary key(Hotel_ID)

);

create table Room

( Room_ID integer,

Room_Type varchar(100),

Price real,

Hotel_ID integer,

constraint pk_Room primary key(Room_ID),

foreign key(Hotel_ID) references Hotel(Hotel_ID)

);

create table Guest

( Guest_ID integer,

First_Name varchar(100),

Middle_Name varchar(100),

Last_Name varchar(100),

Phone varchar(15),

Email varchar(100),
```

```

constraint pk_Guest primary key(Guest_ID)

);

create table Staff

( Staff_ID integer,

First_Name varchar(100),

Last_Name varchar(100),

PhoneNo varchar(15),

Hotel_ID integer,

constraint pk_Staff primary key(Staff_ID),

foreign key(Hotel_ID) references Hotel(Hotel_ID)

);

create table Reservation

( Reservation_ID integer,

Check_IN date,

Check_Out date,

Status varchar(30),

Guest_ID integer,

Room_ID integer,

constraint pk_Reservation primary key(Reservation_ID),

foreign key(Room_ID) references Room(Room_ID),

foreign key(Guest_ID) references Guest(Guest_ID)

);

create table Payment

( Payment_ID integer,

Amount real,

payment_Date Date,

Payment_Status varchar(30),

Reservation_ID integer,

constraint pk_Payment primary key(Payment_ID),

foreign key(Reservation_ID) references Reservation(Reservation_ID)

);

```

8.2 DML – Data Manipulation Language

Sample DML Statements

```
INSERT INTO Hotel
(Hotel_ID, Hotel_name, Location, Contact_no)
VALUES
(1, 'Grand Palace', 'Hyderabad', '9876500001');
INSERT INTO Room
(Room_ID, Room_Type, Price, Hotel_ID)
VALUES
(111, 'Deluxe', 3800.00, 8);
INSERT INTO Guest
(Guest_ID, First_Name, Middle_Name, Last_Name, Phone, Email)
VALUES
(5, 'Kiran', 'Kumar', 'Rao', '9876100005', 'kiran@gmail.com');
INSERT INTO Staff
(Staff_ID, First_Name, Last_Name, PhoneNo, Hotel_ID)
VALUES
(208, 'Vijay', 'Gupta', '9123400008', 8);
INSERT INTO Reservation
(Reservation_ID, Check_IN, Check_Out, Status, Guest_ID, Room_ID)
VALUES
(1007, DATE '2026-04-02', DATE '2026-04-05', 'Completed', 7, 107);
INSERT INTO Payment
(Payment_ID, Amount, payment_Date, Payment_Status, Reservation_ID)
VALUES
(5008, 18000.00, DATE '2026-04-15', 'Paid', 1008);
```

8.3 DQL – Data Query Language

```
SELECT * FROM Staff;
```

```
SELECT Staff_ID, First_Name, Last_Name, PhoneNo
FROM Staff
WHERE Hotel_ID = 8;
```

```
SELECT Hotel_ID, Hotel_name, Location, Contact_no
FROM Hotel
ORDER BY Hotel_name;
```

```
SELECT s.First_Name, s.Last_Name, h.Hotel_name
FROM Staff s
JOIN Hotel h ON s.Hotel_ID = h.Hotel_ID;
```

```
SELECT g.First_Name, g.Last_Name, r.Room_Type, res.Check_IN, res.Check_Out
FROM Guest g
JOIN Reservation res ON g.Guest_ID = res.Guest_ID
JOIN Room r ON res.Room_ID = r.Room_ID;
```

9. SQL QUERIES & SCREENSHOTS

The following query set is prepared to satisfy the documentation requirement for a 30–40 query SQL section. The examples cover selection, filtering, joins, aggregation, subqueries, updates, deletion, and reporting. Exact outputs depend on the records inserted in the final project database.

Q1. Display all hotels

```
SELECT * FROM Hotel;
```

Output:

HOTEL_ID	HOTEL_NAME	LOCATION	CONTACT_NO
1	Grand Palace	Hyderabad	9876500001
2	Royal Residency	Vijayawada	9876500002
3	Green Valley	Visakhapatnam	9876500003
4	Sunrise Hotel	Chennai	9876500004
5	Ocean View	Goa	9876500005
6	City Comfort	Bangalore	9876500006
7	Lake View	Udaipur	9876500007
8	Golden Inn	Mumbai	9876500008
9	Blue Diamond	Pune	9876500009
10	Pearl Residency	Kochi	9876500010
11	Royal Orchid	Delhi	9876500011
12	Silver Star	Kolkata	9876500012
13	Comfort Suites	Jaipur	9876500013
14	Elite Stay	Mysore	9876500014
15	Paradise Hotel	Pondicherry	9876500015

Q2. Display all rooms

```
SELECT * FROM Room;
```

Output:

ROOM_ID	ROOM_TYPE	PRICE	HOTEL_ID
101	Single	1500	1
102	Double	2500	1
103	Deluxe	3500	2
104	Suite	5000	2
105	Single	1800	3
106	Double	2800	3
107	Deluxe	4000	4
108	Suite	6000	5
109	Single	1600	6
110	Double	2700	7
111	Deluxe	3800	8
112	Suite	5500	9
113	Single	1700	10
114	Double	2900	11
115	Deluxe	4200	12

Q3. Display all guests

SELECT * FROM Guest;

Output:

GUEST_ID	FIRST_NAME	MIDDLE_NAME	LAST_NAME	PHONE	EMAIL
1	Rahul	Kumar	Sharma	9876100001	rahul@gmail.com
2	Priya	Rani	Patel	9876100002	priya@gmail.com
3	Arjun	Raj	Reddy	9876100003	arjun@gmail.com
4	Sneha	Devi	Verma	9876100004	sneha@gmail.com
5	Kiran	Kumar	Rao	9876100005	kiran@gmail.com
6	Ananya	Sri	Mehta	9876100006	ananya@gmail.com
7	Vikram	Raj	Singh	9876100007	vikram@gmail.com
8	Neha	Rani	Gupta	9876100008	neha@gmail.com
9	Rohit	Kumar	Das	9876100009	rohit@gmail.com
10	Swathi	Devi	Naidu	9876100010	swathi@gmail.com
11	Aditya	Raj	Kapoor	9876100011	aditya@gmail.com
12	Meghana	Sri	Joshi	9876100012	meghana@gmail.com
13	Sanjay	Kumar	Iyer	9876100013	sanjay@gmail.com
14	Pooja	Rani	Shah	9876100014	pooja@gmail.com
15	Varun	Kumar	Menon	9876100015	varun@gmail.com

Q4. Display all staff

```
SELECT * FROM Staff;
```

Output:

STAFF_ID	FIRST_NAME	LAST_NAME	PHONENO	HOTEL_ID
201	Amit	Sharma	9123400001	1
202	Ravi	Kumar	9123400002	2
203	Suresh	Reddy	9123400003	3
204	Anil	Patel	9123400004	4
205	Rajesh	Rao	9123400005	5
206	Karthik	Singh	9123400006	6
207	Manoj	Verma	9123400007	7
208	Vijay	Gupta	9123400008	8
209	Naveen	Das	9123400009	9
210	Ramesh	Naidu	9123400010	10
211	Arun	Kapoor	9123400011	11
212	Mahesh	Joshi	9123400012	12
213	Prakash	Iyer	9123400013	13
214	Harish	Shah	9123400014	14
215	Mohan	Menon	9123400015	15

Q5. Display rooms with price greater than 3000

```
SELECT * FROM Room
```

```
WHERE Price > 3000;
```

Output:

ROOM_ID	ROOM_TYPE	PRICE	HOTEL_ID
103	Deluxe	3500	2
104	Suite	5000	2
107	Deluxe	4000	4
108	Suite	6000	5
111	Deluxe	3800	8
112	Suite	5500	9
115	Deluxe	4200	12

Q6. Display guests whose first name starts with A

```
SELECT * FROM Guest
```

```
WHERE First_Name LIKE 'A%';
```

Output:

GUEST_ID	FIRST_NAME	MIDDLE_NAME	LAST_NAME	PHONE	EMAIL
3	Arjun	Raj	Reddy	9876100003	arjun@gmail.com
6	Ananya	Sri	Mehta	9876100006	ananya@gmail.com
11	Aditya	Raj	Kapoor	9876100011	aditya@gmail.com

Q7. Guests sorted by first name

```
SELECT * FROM Guest
```

```
ORDER BY First_Name;
```

Output:

GUEST_ID	FIRST_NAME	MIDDLE_NAME	LAST_NAME	PHONE	EMAIL
11	Aditya	Raj	Kapoor	9876100011	aditya@gmail.com
6	Ananya	Sri	Mehta	9876100006	ananya@gmail.com
3	Arjun	Raj	Reddy	9876100003	arjun@gmail.com
5	Kiran	Kumar	Rao	9876100005	kiran@gmail.com
12	Meghana	Sri	Joshi	9876100012	meghana@gmail.com
8	Neha	Rani	Gupta	9876100008	neha@gmail.com
14	Pooja	Rani	Shah	9876100014	pooja@gmail.com
2	Priya	Rani	Patel	9876100002	priya@gmail.com
1	Rahul	Kumar	Sharma	9876100001	rahul@gmail.com
9	Rohit	Kumar	Das	9876100009	rohit@gmail.com
13	Sanjay	Kumar	Iyer	9876100013	sanjay@gmail.com
4	Sneha	Devi	Verma	9876100004	sneha@gmail.com
10	Swathi	Devi	Naidu	9876100010	swathi@gmail.com
15	Varun	Kumar	Menon	9876100015	varun@gmail.com
7	Vikram	Raj	Singh	9876100007	vikram@gmail.com

Q8. Display rooms sorted by price in descending order

```
SELECT * FROM Room
```

```
ORDER BY Price DESC;
```

Output:

ROOM_ID	ROOM_TYPE	PRICE	HOTEL_ID
108	Suite	6000	5
112	Suite	5500	9
104	Suite	5000	2
115	Deluxe	4200	12
107	Deluxe	4000	4
111	Deluxe	3800	8
103	Deluxe	3500	2
114	Double	2900	11
106	Double	2800	3
110	Double	2700	7
102	Double	2500	1
105	Single	1800	3
113	Single	1700	10
109	Single	1600	6
101	Single	1500	1

Q9. Display guests who have an email address

```
SELECT Guest_ID, First_Name, Last_Name, Email
```

```
FROM Guest
```

```
WHERE Email IS NOT NULL;
```

Output:

GUEST_ID	FIRST_NAME	LAST_NAME	EMAIL
1	Rahul	Sharma	rahul@gmail.com
2	Priya	Patel	priya@gmail.com
3	Arjun	Reddy	arjun@gmail.com
4	Sneha	Verma	sneha@gmail.com
5	Kiran	Rao	kiran@gmail.com
6	Ananya	Mehta	ananya@gmail.com
7	Vikram	Singh	vikram@gmail.com
8	Neha	Gupta	neha@gmail.com
9	Rohit	Das	rohit@gmail.com
10	Swathi	Naidu	swathi@gmail.com
11	Aditya	Kapoor	aditya@gmail.com
12	Meghana	Joshi	meghana@gmail.com
13	Sanjay	Iyer	sanjay@gmail.com
14	Pooja	Shah	pooja@gmail.com
15	Varun	Menon	varun@gmail.com

Q10. Display all rooms with their hotel names

```
SELECT r.Room_ID, r.Room_Type, r.Price, h.Hotel_name
```

```
FROM Room r
```

```
JOIN Hotel h
```

```
ON r.Hotel_ID = h.Hotel_ID;
```

Output:

ROOM_ID	ROOM_TYPE	PRICE	HOTEL_NAME
101	Single	1500	Grand Palace
102	Double	2500	Grand Palace
103	Deluxe	3500	Royal Residency
104	Suite	5000	Royal Residency
105	Single	1800	Green Valley
106	Double	2800	Green Valley
107	Deluxe	4000	Sunrise Hotel
108	Suite	6000	Ocean View
109	Single	1600	City Comfort
110	Double	2700	Lake View
111	Deluxe	3800	Golden Inn
112	Suite	5500	Blue Diamond
113	Single	1700	Pearl Residency
114	Double	2900	Royal Orchid
115	Deluxe	4200	Silver Star

Q11. Display staff with their hotel names

```
SELECT s.Staff_ID, s.First_Name, s.Last_Name, h.Hotel_name
FROM Staff s
JOIN Hotel h
ON s.Hotel_ID = h.Hotel_ID;
```

Output:

STAFF_ID	FIRST_NAME	LAST_NAME	HOTEL_NAME
201	Amit	Sharma	Grand Palace
202	Ravi	Kumar	Royal Residency
203	Suresh	Reddy	Green Valley
204	Anil	Patel	Sunrise Hotel
205	Rajesh	Rao	Ocean View
206	Karthik	Singh	City Comfort
207	Manoj	Verma	Lake View
208	Vijay	Gupta	Golden Inn
209	Naveen	Das	Blue Diamond
210	Ramesh	Naidu	Pearl Residency
211	Arun	Kapoor	Royal Orchid
212	Mahesh	Joshi	Silver Star
213	Prakash	Iyer	Comfort Suites
214	Harish	Shah	Elite Stay
215	Mohan	Menon	Paradise Hotel

Q12. Display hotel name and location

```
SELECT Hotel_name, Location
```

```
FROM Hotel;
```

Output:



HOTEL_NAME	LOCATION
Grand Palace	Hyderabad
Royal Residency	Vijayawada
Green Valley	Visakhapatnam
Sunrise Hotel	Chennai
Ocean View	Goa
City Comfort	Bangalore
Lake View	Udaipur
Golden Inn	Mumbai
Blue Diamond	Pune
Pearl Residency	Kochi
Royal Orchid	Delhi
Silver Star	Kolkata
Comfort Suites	Jaipur
Elite Stay	Mysore
Paradise Hotel	Pondicherry

Q13. Display reservation details with guest names

```

SELECT r.Reservation_ID, g.First_Name,g.Last_Name, r.Check_IN, r.Check_Out, r.Status
FROM Reservation r
JOIN Guest g
ON r.Guest_ID = g.Guest_ID;

```

Output:

RESERVATION_ID	FIRST_NAME	LAST_NAME	CHECK_IN	CHECK_OUT	STATUS
1001	Rahul	Sharma	05-JAN-26	08-JAN-26	Completed
1002	Priya	Patel	10-JAN-26	12-JAN-26	Completed
1003	Arjun	Reddy	01-FEB-26	04-FEB-26	Completed
1004	Sneha	Verma	15-FEB-26	18-FEB-26	Completed
1005	Kiran	Rao	03-MAR-26	06-MAR-26	Completed
1006	Ananya	Mehta	10-MAR-26	13-MAR-26	Completed
1007	Vikram	Singh	02-APR-26	05-APR-26	Completed
1008	Neha	Gupta	15-APR-26	18-APR-26	Completed
1009	Rohit	Das	01-MAY-26	04-MAY-26	Completed
1010	Swathi	Naidu	12-MAY-26	15-MAY-26	Completed
1011	Aditya	Kapoor	05-JUN-26	08-JUN-26	Confirmed
1012	Meghana	Joshi	15-JUN-26	18-JUN-26	Confirmed
1013	Sanjay	Iyer	01-JUL-26	04-JUL-26	Confirmed
1014	Pooja	Shah	10-JUL-26	13-JUL-26	Confirmed
1015	Varun	Menon	01-AUG-26	04-AUG-26	Confirmed

Q14. Display reservation details with room information

```

SELECT r.Reservation_ID,

       g.First_Name,

       g.Last_Name,

       rm.Room_ID,

       rm.Room_Type,

       r.Status

FROM Reservation r

JOIN Guest g

ON r.Guest_ID = g.Guest_ID

JOIN Room rm

ON r.Room_ID = rm.Room_ID;

```

Output:

RESERVATION_ID	FIRST_NAME	LAST_NAME	ROOM_ID	ROOM_TYPE	STATUS
1001	Rahul	Sharma	101	Single	Completed
1002	Priya	Patel	102	Double	Completed
1003	Arjun	Reddy	103	Deluxe	Completed
1004	Sneha	Verma	104	Suite	Completed
1005	Kiran	Rao	105	Single	Completed
1006	Ananya	Mehta	106	Double	Completed
1007	Vikram	Singh	107	Deluxe	Completed
1008	Neha	Gupta	108	Suite	Completed
1009	Rohit	Das	109	Single	Completed
1010	Swathi	Naidu	110	Double	Completed
1011	Aditya	Kapoor	111	Deluxe	Confirmed
1012	Meghana	Joshi	112	Suite	Confirmed
1013	Sanjay	Iyer	113	Single	Confirmed
1014	Pooja	Shah	114	Double	Confirmed
1015	Varun	Menon	115	Deluxe	Confirmed

Q15. Count rooms by room type

```

SELECT Room_Type,
COUNT(*) AS Room_Count
FROM Room
GROUP BY Room_Type;

```

Output:

ROOM_TYPE	ROOM_COUNT
Suite	3
Single	4
Deluxe	4
Double	4

```

FROM Payment
GROUP BY Payment_Status;

```

Output:

PAYMENT_STATUS	TOTAL_AMOUNT
Paid	91700
Pending	54300



11. CONCLUSION

The Hotel Reservation Management System provides a structured relational database solution for managing essential hotel operations. The system organizes hotels, rooms, guests, staff, reservations, and payments into related tables. Primary keys and foreign keys are used to establish relationships and maintain data integrity. SQL queries demonstrate data insertion, retrieval, filtering, sorting, joining, aggregation, and reporting. The database provides an efficient foundation for managing hotel reservations and payment information and can be further extended with additional hotel management features.

12. FUTURE ENHANCEMENTS

- Role-based access control for administrators, managers, receptionists, and other staff.
- Online room booking and cancellation functionality.
- Automated payment and billing management.
- Guest check-in and check-out management.
- Room availability tracking in real time.
- Automated reservation confirmations and notifications.
- Customer feedback and review management.
- Detailed revenue and occupancy reports.
- Backup, recovery, audit logging, and improved database security.
- Integration with a web or mobile application for easier hotel management.



REFERENCES

1. Korth, H. F., Silberschatz, A., and Sudarshan, S., Database System Concepts, McGraw Hill.
2. Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.
3. Silberschatz, A., Korth, H. F., and Sudarshan, S., *Database System Concepts*, 7th Edition, McGraw Hill.
4. Elmasri, R. and Navathe, S. B., *Fundamentals of Database Systems*, 7th Edition, Pearson.
5. Oracle Corporation, *Oracle Database 10g Documentation*, Oracle Corporation.
6. Oracle Corporation, *Oracle Database SQL Reference*, Oracle Corporation.
7. Oracle Corporation, *Oracle Database Concepts*, Oracle Corporation.
8. Ramakrishnan, R. and Gehrke, J., *Database Management Systems*, 3rd Edition, McGraw Hill.
9. Date, C. J., *An Introduction to Database Systems*, 8th Edition, Pearson.
10. Connolly, T. and Begg, C., *Database Systems: A Practical Approach to Design, Implementation, and Management*, Pearson.
11. PostgreSQL Global Development Group, *PostgreSQL Documentation*, PostgreSQL.
12. MySQL, *MySQL 8.0 Reference Manual*, Oracle Corporation.

